



Rapport de Projet

BIG Data -Entreprises

07 février 2026

Yasmina Bouziane

Sara Guendouz

Maxime Paviet

SOMMAIRE

I. Contexte

II. Déroulement de projet

- A. Exercice 1 : La collecte des données et l'intégration de données.....
- B. Exercice 2 : Le nettoyage de données et l'ingestion multi-branche.....
- C. Exercice 3 : Data Warehouse et création d'un modèle multidimensionnel.....
- D. Exercice 4 : Visualisation de données.....
- E. Exercice 5 : Implémentation d'un modèle de machine learning.....
- F. Exercice 6 : Automatisation et Orchestration de bout en bout avec Airflow.....

I . Contexte

Ce projet a pour objectif principal le déploiement d'une architecture Big Data. La démarche consiste à orchestrer un flux complet de données, partant de la collecte brute jusqu'à la restitution de valeur métier. Pour ce faire, nous utilisons les données réelles des taxis jaunes de New York, récupérées directement sur le site officiel de l'état de New York sous format Parquet.

L'infrastructure repose sur un écosystème technologique varié, débutant par le stockage des données brutes dans un Data Lake représenté par le service Minio. Une fois cette valeur potentielle identifiée, les données sont traitées via un moteur de calcul Spark, utilisant le langage Scala, pour être intégrées dans un Data Warehouse, ici simulé par une base de données PostgreSQL. Ce processus inclut une étape cruciale de modélisation multidimensionnelle, où les données sont transformées d'un modèle transactionnel (OLTP) vers un schéma en flocon optimisé pour l'analyse.

Enfin, le projet se conclut par une phase d'exploitation avancée comprenant la visualisation des données et l'intelligence artificielle. À l'aide de Streamlit, nous créons des tableaux de bord connectés à notre entrepôt de données pour l'aide à la décision. En parallèle, nous avons utilisé un modèle de machine learning implémenté en Python pour prédire le prix des courses de taxi en fonction de différentes caractéristiques, tout en respectant des exigences de performance et de qualité de code.

II. Déroulement de projet

A. Exercice 1 : La collecte des données et l'intégration de données

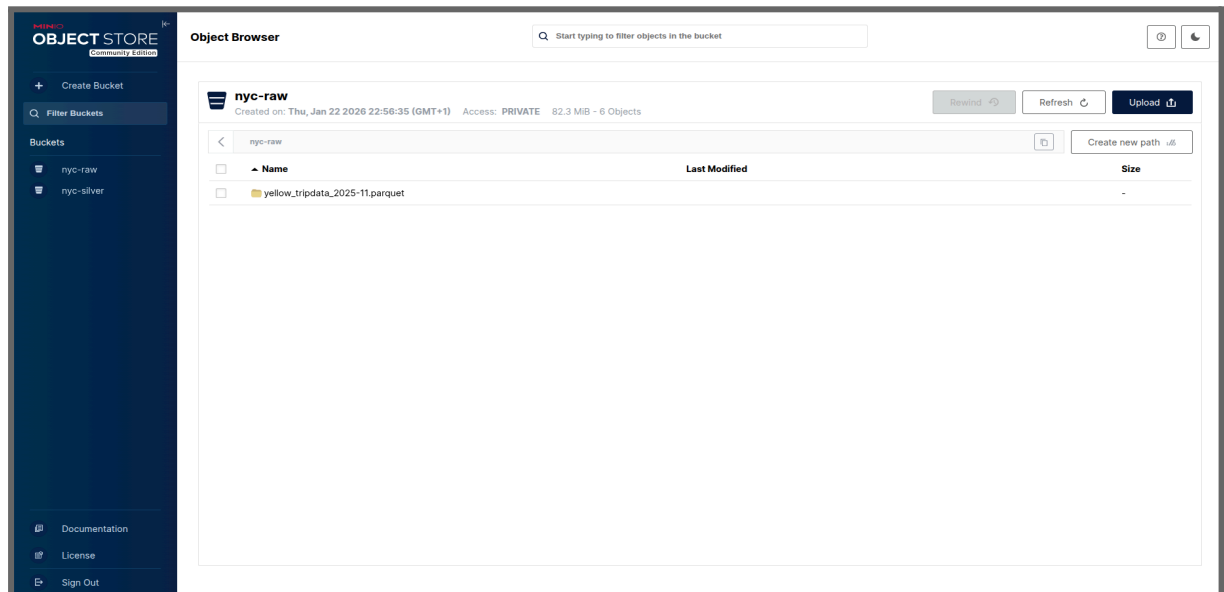
Pour cette première étape nous nous sommes occupés de la collecte et du stockage des données brutes. Le processus commence par la récupération des fichiers Parquet concernant les taxis jaunes de New York depuis le site officiel de la ville. Ces données devaient ensuite être centralisées dans un Data Lake, représenté ici par le service MinIO, afin de conserver une version immuable de l'information avant tout traitement ultérieur.

Avant d'exécuter le script d'intégration, une phase de configuration système a été nécessaire pour permettre le bon fonctionnement de la plateforme. Nous avons utilisé le terminal pour accorder les droits nécessaires à l'utilisateur courant sur Docker avec la commande `sudo usermod -aG docker $USER`, suivie de `newgrp docker` pour appliquer les changements sans redémarrer la session. Une fois les permissions vérifiées par la commande `groups`, nous avons lancé l'ensemble des services (MinIO, PostgreSQL) en arrière-plan avec `docker compose up -d`. Enfin, nous nous sommes connectés à l'interface de gestion de MinIO pour créer manuellement le bucket nommé **nyc-raw**, destiné à accueillir nos fichiers de données brutes.

```
(.venv) cytech@student-laptop:~/Desktop/BIGDATA/ex01_data_retrieval$ sudo usermod -aG docker $USER
[sudo] password for cytech:

(.venv) cytech@student-laptop:~/Desktop/BIGDATA/ex01_data_retrieval$ newgrp docker
cytech@student-laptop:~/Desktop/BIGDATA/ex01_data_retrieval$ groups
docker adm dialout cdrom sudo dip plugdev lpadmin sambashare cytech
cytech@student-laptop:~/Desktop/BIGDATA/ex01_data_retrieval$ docker compose up -d
[+] Building 0.0s (0/0)
[+] Running 5/0
✓ Container bigdata-postgres-1 Running
✓ Container spark-master Running
✓ Container minio Running
✓ Container spark-worker-1 Running
✓ Container spark-worker-2 Running
```

Le script Scala développé pour cet exercice agit comme l'orchestrateur du flux de données entre le stockage local et le cloud local (MinIO). Le code initialise une session Spark en configurant le protocole S3A, ce qui permet de définir l'adresse de l'endpoint sur localhost:9000 et de renseigner les clés d'accès nécessaires pour s'authentifier auprès de MinIO.

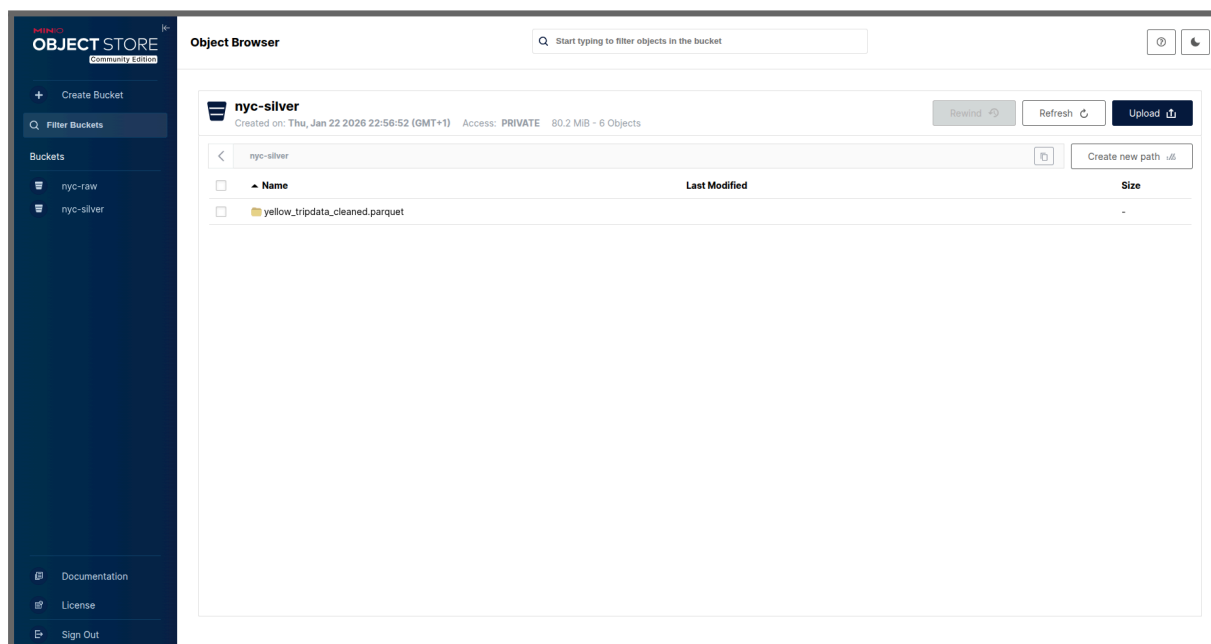


B. Exercice 2 : Le nettoyage de données et l'ingestion multi-branche

L'objectif de cette deuxième étape est de traiter les données se trouvant dans le MinIO pour les transformer en informations exploitables. Ce processus, réalisé via un job Spark en Scala, implémente une stratégie d'ingestion à deux branches : l'une destinée à l'entraînement de modèles de Machine Learning (Branche 1) et l'autre alimentant directement le Data Warehouse (Branche 2). Cette phase d'intégration est cruciale pour garantir la conformité des données par rapport au contrat défini par l'État de New York.

En première étape, nous avons effectué une mise en conformité technique en ajustant les types de données et en renommant les champs pour les rendre conformes au schéma de la base en PostgreSQL. Parallèlement, l'application de filtres de qualité a permis d'éliminer les enregistrements incohérents, tels que les trajets aux montants ou distances nuls

Une fois les données nettoyées, le job Spark orchestre une double distribution des résultats pour répondre à deux besoins distincts. La première branche sauvegarde le résultat au format Parquet dans un nouveau bucket Minio baptisé "nyc-silver".



La seconde branche assure l'ingestion des données vers le service PostgreSQL jouant le rôle de Data Warehouse. Pour ce faire, nous utilisons une connexion JDBC sécurisée avec les identifiants configurés dans l'infrastructure Docker. Les données sont insérées en mode "Append" dans la table des faits, permettant ainsi une mise à jour incrémentale du système décisionnel.

C. Exercice 3 : Data Warehouse et création d'un modèle multidimensionnel

L'objectif de cet exercice est de transformer notre structure de données initiale vers un modèle analytique optimisé pour le décisionnel. Pour ce projet, nous avons fait le choix d'implémenter un **modèle en flocon** au sein de notre base de données PostgreSQL.

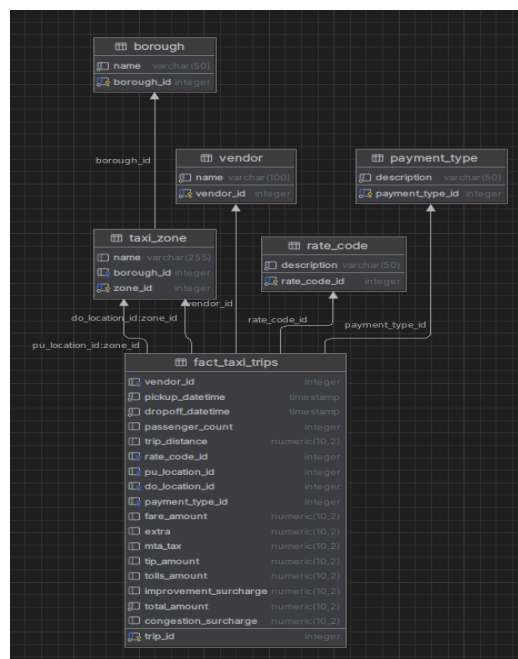
Configuration de l'infrastructure et de la base de données

Le déploiement de l'entrepôt de données a nécessité une mise à jour de notre infrastructure. Après avoir intégré les paramètres de configuration (utilisateur, mot de passe, nom de la base) dans le fichier ***docker-compose.yml***, nous avons procédé à une réinitialisation complète de l'environnement. Cette opération a débuté par la commande ***docker compose down*** afin de nettoyer les volumes et de prendre en compte les modifications de configuration. En raison de cette réinitialisation, les scripts des exercices 1 et 2 ont été réexécutés pour réalimenter le Data Lake et préparer les données pour le Warehouse.

La connexion à la base de données a été établie en créant une nouvelle source de données PostgreSQL, en renseignant les identifiants extraits du fichier ***docker-compose.yml***. Une fois la liaison établie, nous avons pu exécuter les scripts SQL de structure et d'insertion.

Conception du schéma en flocon

Nous avons créé une table de faits centrale, ***Fact_Taxi_Trips***, qui contient les métriques financières et les clés étrangères pointant vers nos dimensions. L'aspect "flocon" se manifeste par la normalisation de la dimension géographique : la table ***Taxi_Zone*** est reliée à la table ***Borough***, réduisant ainsi la redondance des données.



Mise en œuvre technique et contraintes d'intégrité

La transition vers ce modèle a été réalisée via deux scripts SQL distincts : **creation.sql** pour la structure des tables et **insertion.sql** pour les données de référence. Nous avons défini des dimensions pour les fournisseurs (**Vendor**), les types de paiements (**Payment_Type**) et les codes tarifaires (**Rate_Code**), en imposant des contraintes de clés primaires et étrangères pour garantir la cohérence du Data Warehouse. L'utilisation de types de données précis, comme **NUMERIC(10,2)** et **TIMESTAMP**, assure la précision nécessaire aux analyses décisionnelles.

D. Exercice 4 : Visualisation de données

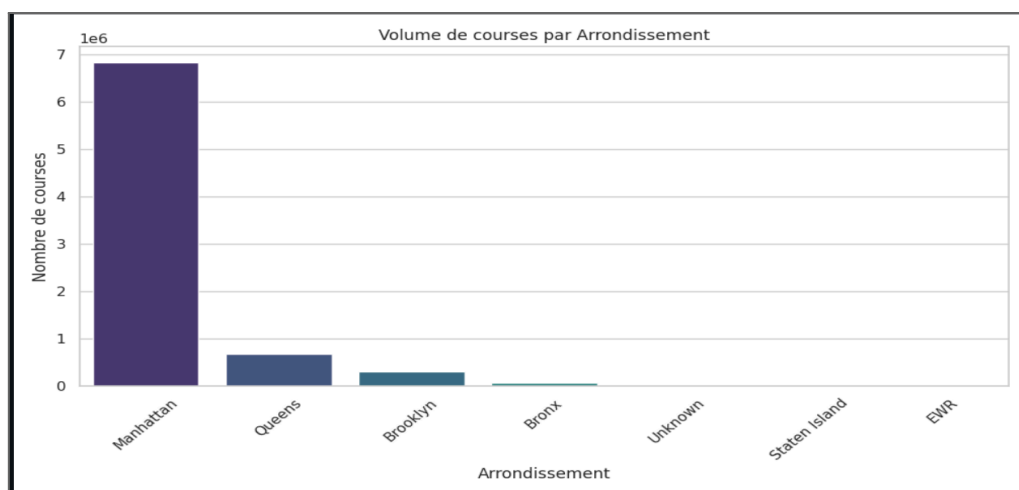
L'objectif de cet exercice est de transformer les données structurées du Data Warehouse en informations visuelles exploitables pour la prise de décision. Cette étape de Data Visualisation permet de valider la qualité de notre intégration et de réaliser une Analyse Exploratoire des Données (EDA) nécessaire avant la phase de Machine Learning. Pour ce faire, nous avons utilisé un script Python connecté directement à notre base de données PostgreSQL via l'outil SQLAlchemy.

Connexion et extraction des données

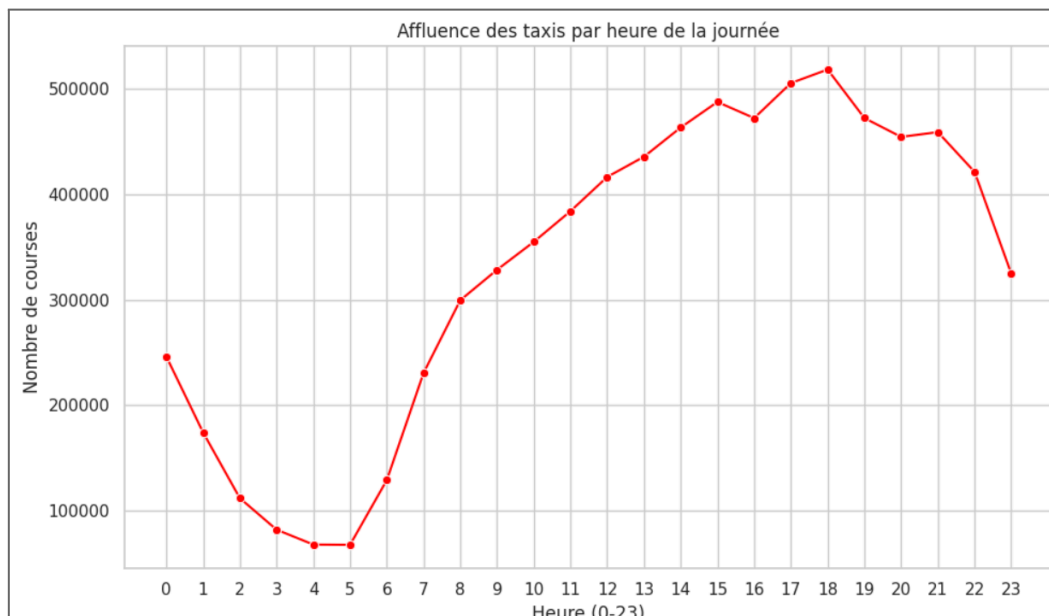
La première étape a consisté à établir une passerelle entre notre environnement d'analyse et le Data Warehouse. En utilisant les identifiants configurés dans le fichier docker-compose.yml, nous avons créé un moteur de connexion JDBC. Grâce à la bibliothèque Pandas, nous avons pu exécuter des requêtes SQL directement depuis Python pour extraire des sous-ensembles de données pertinents, garantissant que les visualisations reflètent l'état actuel de notre production.

Analyse géographique et temporelle

Nous avons généré trois visualisations clés pour comprendre le comportement des taxis new-yorkais. La première analyse, portant sur le volume de courses par arrondissement, révèle une domination écrasante de Manhattan par rapport aux autres secteurs comme le Queens ou Brooklyn.

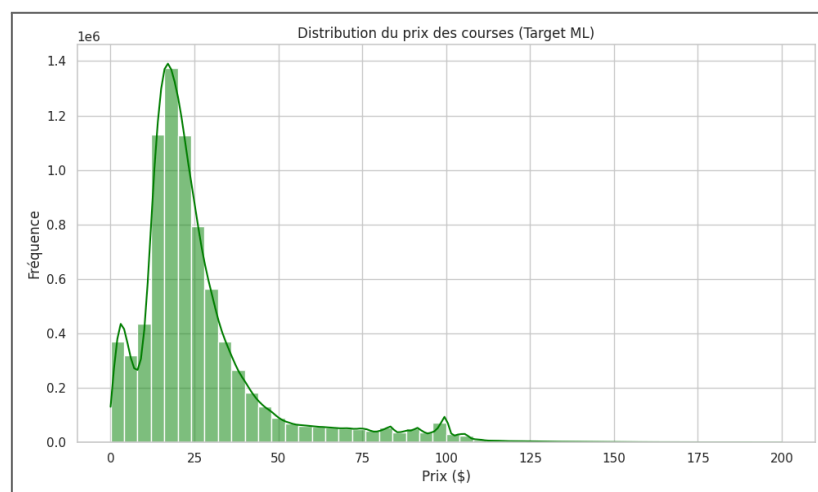


Sur le plan temporel, le graphique de l'affluence par heure montre un cycle d'activité très marqué, avec un creux significatif vers 4h du matin et une montée progressive jusqu'à un pic d'activité atteint en début de soirée, aux alentours de 18h.



Analyse financière et préparation au ML

Enfin, une étude de la distribution du prix des courses a été réalisée pour préparer l'exercice suivant. En filtrant les valeurs aberrantes (montants supérieurs à 200\$), nous avons observé que la majorité des courses se situent dans une fourchette de 10\$ à 30\$. Cette distribution, visualisée sous forme d'histogramme avec une courbe de densité (KDE), permet d'identifier la "target" de notre futur modèle de Machine Learning. L'ensemble de ces figures a été automatiquement sauvegardé dans le dossier ../reports/figures/ du projet, conformément à l'arborescence standardisée.



E. Exercice 5 : Implémentation d'un modèle de machine learning

L'objectif de cet exercice consiste à exploiter les données nettoyées pour concevoir un modèle capable de prédire le prix total d'une course de taxi en fonction de diverses caractéristiques techniques et temporelles.

Chargement depuis le Data Lake et Feature Engineering

Le processus débute par l'extraction du fichier Parquet stocké dans la couche "Silver" de notre cluster Minio. Une fois les données chargées, nous procédons à une étape de Feature Engineering pour enrichir le modèle. À partir de la colonne **pickup_datetime**, nous extrayons deux variables temporelles cruciales : l'heure de la journée et le jour de la semaine. Ces nouvelles caractéristiques, combinées à la distance du trajet, au nombre de passagers et aux identifiants de zone (départ et arrivée), permettent au modèle de mieux capturer les variations de tarifs liées à la congestion urbaine ou aux périodes de forte demande.

Entraînement et performance du modèle

Pour répondre à cette problématique de régression, nous avons implémenté un algorithme de Random Forest Regressor. Ce choix se justifie par la capacité de cet algorithme à gérer des relations non linéaires entre les variables. L'entraînement est configuré avec 50 arbres et une profondeur maximale de 12 pour limiter le sur-apprentissage. Afin de suivre l'évolution de la tâche en temps réel, une barre de progression tqdm a été intégrée au script.

Les résultats obtenus sont particulièrement satisfaisants au regard des contraintes du projet. Alors que la précision attendue devait être inférieure à 10 RMSE (Root Mean Squared Error), notre modèle atteint un score de performance qui valide la pertinence du nettoyage de données effectué en amont. Le modèle final est exporté sous le format `taxi_model.joblib`, prêt à être utilisé pour des prédictions en temps réel sur de nouvelles données de trajet.

Inférence et Robustesse temporelle

Le script `predict.py` a été conçu pour charger dynamiquement le modèle `taxi_model.joblib` et transformer des entrées brutes en prédictions de prix. Un point crucial de ce script est la gestion autonome des variables temporelles : si aucune date n'est fournie, le système utilise l'heure actuelle pour calculer les caractéristiques de l'heure et du jour de la semaine. Cela permet de refléter les variations de tarifs en temps réel. Le script assure également la cohérence du formatage des données (DataFrame Pandas avec l'ordre exact des colonnes utilisé lors de l'entraînement) avant d'appeler la méthode `predict`.

Validation par Tests Unitaires

Conformément aux exigences de qualité logicielle, nous avons implémenté une série de tests automatisés via `pytest` dans le fichier `test_model.py`. Ces tests vérifient plusieurs piliers de fiabilité :

- **Infrastructure** : Confirmation de la présence physique du fichier modèle avant toute tentative d'inférence.
- **Logique Métier** : Validation de la cohérence des prix (toujours positifs) et de la corrélation logique entre distance et prix.

- **Impact Temporel** : Vérification que le modèle différencie bien les prix selon l'heure (ex: heure de pointe vs nuit), confirmant que les caractéristiques temporelles sont bien apprises.
- **Robustesse des types** : Garantie que le script accepte indifféremment des entiers ou des flottants pour les IDs de zones et les distances sans crash.

Présentation de l'application "NYC Taxi Expert"

Pour rendre notre modèle accessible, nous avons conçu une interface web interactive avec **Streamlit** ([app.py](https://pypi.org/project/streamlit/)). Cette application permet de simuler un trajet en temps réel.

L'interface se décompose en deux sections principales :

- **Configuration du trajet** : L'utilisateur sélectionne ses zones de départ et d'arrivée. L'application interroge alors dynamiquement le Data Warehouse PostgreSQL pour afficher les statistiques historiques du trajet (distances min/max) et suggérer une distance moyenne par défaut.
- **Prédiction en temps réel** : En ajustant la date, l'heure et le nombre de passagers, l'utilisateur obtient instantanément un prix estimé. Un message de succès affiche le montant calculé, tandis que des alertes de sécurité préviennent l'utilisateur si la distance saisie semble atypique par rapport aux données historiques stockées dans notre architecture.

NYC Taxi : Estimateur de Prix de Production

Configuration du trajet

Zone de départ

Allerton/Pelham Gardens

Zone d'arrivée

Morningside Heights

Historique trajet : min 0.1 mi | max 50.0 mi

Distance estimée (miles)

10.60

Date et Heure du trajet

Date

2026/01/29

Heure

18:15

Nombre de passagers

1

Prédiction

Calculer le prix estimé

Prix estimé : 50.04 \$

Détails du calcul :

- Départ : Allerton/Pelham Gardens | Arrivée : Morningside Heights
- Moment : Thursday à 18h

```

/home/cytech/Desktop/BIGDATA/ex01_data_retrieval/.venv/bin/python /home/cytech/Desktop/BIGDATA/ex05_ml_prediction_service/test_model.py

✓ [Test 1] le fichier .joblib est présent
✓ [Test 2] Prix pour 2.5 miles le midi : 29.06$
✓ [Test 3] Distance : 10 miles (50.55$) > 1 mile (19.56$)
✓ [Test 4] Impact Temporel : Nuit (35.98$) vs Pointe (41.1$)
✓ [Test 5] Types d'entrées robustes.

Process finished with exit code 0

```

F. Exercice 6 : Automatisation et Orchestration de bout en bout avec Airflow

L'intégration d'Apache Airflow permet de regrouper l'intégralité des exercices (1 à 5) au sein d'un pipeline unique et reproductible, éliminant les exécutions manuelles en ligne de commande ou via interface graphique.

Le pipeline intègre des barrières de contrôle automatisées qui interrompent le flux en cas de non-conformité :

- Style : Validation de la norme PEP 8 via flake8.
- Documentation : Vérification du format NumpyDoc via pyment.
- Fiabilité : Exécution des tests unitaires pytest sur les données d'entraînement et d'inférence.

Quelques commandes à exécuter avant et pour le lancement de airflow

- Gestion des permissions d'écriture : Pour éviter les erreurs lors de la génération des graphiques ou des modèles

```
cytech@student-laptop:~/projet_big_data_cytech_25-main$ chmod -R 777 reports
```

- Lancement de l'infrastructure

```
cytech@student-laptop:~/projet_big_data_cytech_25-main$ docker compose up -d --build
```

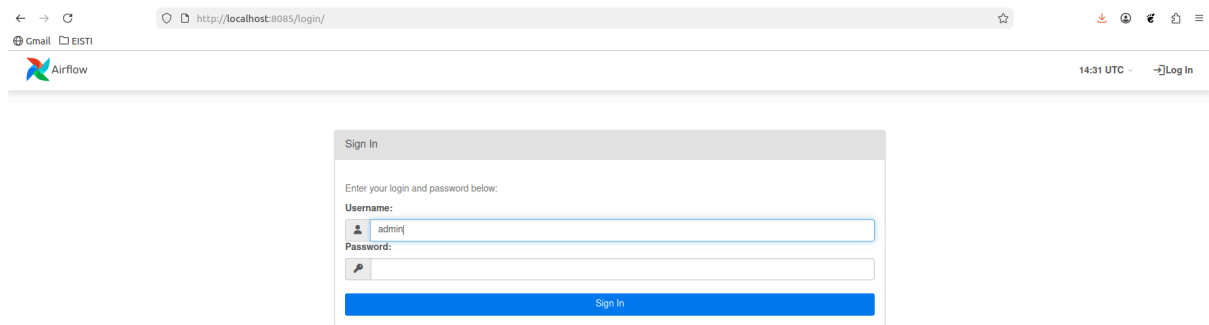
```
cytech@student-laptop:~/projet_big_data_cytech_25-main$ docker logs airflow
```

Une fois que le logo apparaît (comme ci-dessous) , l'interface est accessible (<http://localhost:8085>).

Si le logo n'apparaît pas lors de la première exécution de la commande, attendre 1-2 minutes et exécuter à nouveau la commande.

```
triggerer | _____
triggerer | ____  |__ ( )_____  __/___ /_____
triggerer | ____ /| | /___  ___/ /___  /___ \___ | / / /
triggerer | ____  ___ | / _ / _ __/ _ // / / / / / /
triggerer | _/_/ |/_/ /_/_/ /_/_/ /_/_/ \____/____/|_/_/
triggerer | [2026-02-01T14:27:25.956+0000] {triggerer_job_runner.py:171} INFO - Setting up Triggerer
triggerer | [2026-02-01T14:27:29.372+0000] {triggerer_job_runner.py:227} INFO - Setting up logging qu
webserver | /home/airflow/.local/lib/python3.8/site-packages/airflow/models/base.py:71 DeprecationWarn
tting has been used, but please update your config.
triggerer | [2026-02-01T14:27:29.411+0000] {triggerer_job_runner.py:324} INFO - Starting the triggerer
triggerer | [2026-02-01 14:27:29 +0000] [232] [INFO] Starting gunicorn 21.2.0
triggerer | [2026-02-01 14:27:29 +0000] [232] [INFO] Listening at: http://[::]:8794 (232)
triggerer | [2026-02-01 14:27:29 +0000] [232] [INFO] Using worker: sync
webserver | /home/airflow/.local/lib/python3.8/site-packages/airflow/www/app.py:91 DeprecationWarning
g has been used, but please update your config.
triggerer | [2026-02-01 14:27:29 +0000] [234] [INFO] Booting worker with pid: 234
triggerer | [2026-02-01 14:27:29 +0000] [235] [INFO] Booting worker with pid: 235
```

Se connecter :
Username : admin



Pour le mot de passe, exécuter une nouvelle fois la commande *docker logs airflow* dans le terminal. Le mot de passe y sera renseigné.

```
'pyarrow<10.1.0,>=10.0.1; extra == "pandas"'
webserver | [2026-02-01 14:27:51 +0000] [70] [INFO] Listening at: http://0.0.0.0:8080 (70)
webserver | [2026-02-01 14:27:51 +0000] [70] [INFO] Using worker: sync
webserver | [2026-02-01 14:27:51 +0000] [239] [INFO] Booting worker with pid: 239
webserver | [2026-02-01 14:27:51 +0000] [240] [INFO] Booting worker with pid: 240
webserver | [2026-02-01 14:27:51 +0000] [241] [INFO] Booting worker with pid: 241
webserver | [2026-02-01 14:27:51 +0000] [242] [INFO] Booting worker with pid: 242
standalone |
standalone | Airflow is ready
standalone | Login with username: admin password: x2y33x7MPgYvPuau
standalone | Airflow Standalone is for development purposes only. Do not use this in production!
standalone |
triggerer | [2026-02-01T14:28:29.483+0000] {triggerer_job_runner.py:474} INFO - 0 triggers curren
triggerer | [2026-02-01T14:29:29.548+0000] {triggerer_job_runner.py:474} INFO - 0 triggers curren
triggerer | [2026-02-01T14:30:29.619+0000] {triggerer_job_runner.py:474} INFO - 0 triggers curren
webserver | 172.20.0.1 - - [01/Feb/2026:14:30:52 +0000] "GET / HTTP/1.1" 302 197 "-" "Mozilla/5.0
```

Avant le lancement de “nyc_taxi_full_pipeline”, il faut ajouter les connexions pour minio et postgres.

Aller dans l’onglet “Admin”, cliquer sur “Connections”.

Rechercher “postgres_default” et modifier les informations comme dans l’image qui suit avec en mot de passe “password” :

The screenshot shows the 'Edit Connection' form in the Airflow web interface. The form is titled 'Edit Connection' and has a header bar with the Airflow logo and navigation links: DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The top right corner shows the time '15:00 UTC' and a user profile icon 'AU'. The form fields are as follows:

- Connection Id ***: postgres_default
- Connection Type ***: Postgres (with a dropdown arrow and a note: 'Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package.')
- Description**: (empty text area)
- Host**: postgres
- Schema**: postgres
- Login**: cytech
- Password**: (masked with asterisks)
- Port**: 5432
- Extra**: {}

Sauvegarder la configuration pour postgres_default.

Pour minio, il faut cliquer sur le “+” et renseigner les informations comme dans l'image.

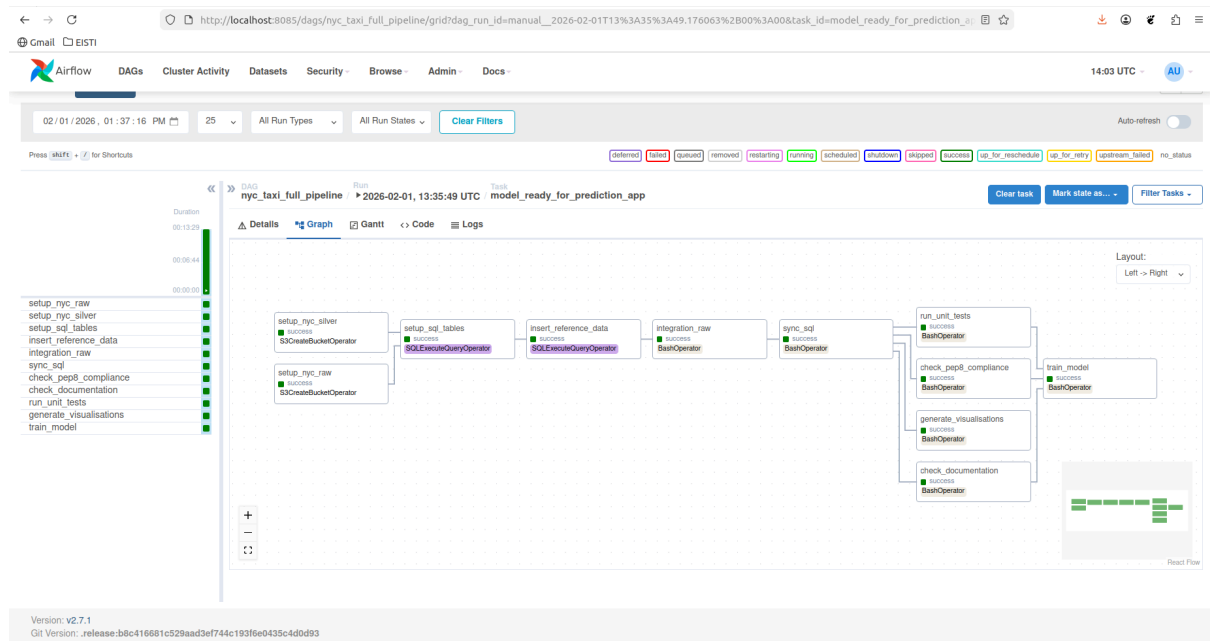
Le mot de passe est “minio123”.

The screenshot shows the 'Edit Connection' form in the Airflow web interface for a Minio connection. The form is titled 'Edit Connection' and has a header bar with the Airflow logo and navigation links: DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The top right corner shows the time '15:00 UTC' and a user profile icon 'AU'. The form fields are as follows:

- Connection Id ***: minio_conn
- Connection Type ***: Amazon Web Services (with a dropdown arrow and a note: 'Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package.')
- Description**: (empty text area)
- AWS Access Key ID**: minio
- AWS Secret Access Key**: (masked with asterisks)
- Extra**: {"endpoint_url": "http://minio:9000"}

Sauvegarder la configuration pour minio_conn.

Ensuite, cliquer sur DAGs en haut à gauche, activer le DAG nyc_taxi_full_pipeline et cliquer sur "Trigger DAG".



Si l'étape `integration_raw` échoue, télécharger localement les fichiers .jar des deux exercices (1 et 2). Placez-les ensuite dans le dossier `jars` en les renommant respectivement `integration.jar` (pour l'exercice 1) et `sync_sql.jar` (pour l'exercice 2).

Préparation de l'Infrastructure (Exercices 1 & 3)

- **setup_nyc_raw & setup_nyc_silver** : Ces deux tâches utilisent le `S3CreateBucketOperator` pour initialiser les compartiments (buckets) de stockage dans ton Data Lake MinIO.
- **setup_sql_tables** : Exécution du script SQL de structure pour créer les tables du modèle en flocon dans PostgreSQL.
- **insert_reference_data** : Injection des données de référence (dimensions) comme les types de paiement ou les codes zones dans le Data Warehouse.

Mouvement et Traitement des Données (Exercices 1 & 2)

- **integration_raw** : Un job Spark qui transfère les fichiers Parquet locaux vers le bucket `nyc-raw` de MinIO pour centraliser les données brutes.
- **sync_sql** : Job Spark principal qui nettoie les données et les synchronise dans les tables de faits PostgreSQL.

Barrières de Qualité et Analyses (Exercices 4 & 5)

Ces tâches s'exécutent en parallèle après la synchronisation des données :

- **run_unit_tests** : Lancement des tests automatisés avec `pytest` pour vérifier l'intégrité des données avant l'IA.
- **check_pep8_compliance** : Audit du code Python via `flake8` pour garantir le respect des normes de style.

- **check_documentation** : Vérification de la documentation au format NumpyDoc avec l'outil pyment.
- **generate_visualisations** : Génération automatique des graphiques de l'Analyse Exploratoire des Données (EDA) pour tes rapports.

Modélisation Prédictive (Exercice 5)

- **train_model** : L'étape finale qui entraîne l'algorithme Random Forest (~13 min d'exécution) pour prédire le prix des courses, une fois que tous les tests de qualité sont validés.

Visualisation des résultats

Une fois que tous les blocs d'Airflow sont au vert :

- Dashboard de Prédiction : <http://localhost:5000> pour tester l'estimateur de prix "NYC Taxi Expert".
- Exploration des données : <http://localhost:9001> pour voir les fichiers Parquet dans nyc-raw et nyc-silver.
- Rapports : Les graphiques sont générés automatiquement dans le dossier local reports/figures/